SIC Transmetro — Diagrama ER, Script de BD y Diagramas UML

Proyecto final · Aseguramiento de la Calidad del Software · 18 de septiembre de 2026

Introducción

Este documento reúne el diseño de datos y el diseño UML del Sistema de Información y Control del Transmetro (SIC Transmetro): el diagrama entidad relación, el script de la base de datos y 15 diagramas UML. Es el entregable de las fases F6 a F8 del proyecto final del curso de Aseguramiento de la Calidad del Software.

El sistema centraliza la información de 7 líneas, 106 estaciones y 210 unidades del Transmetro de la Ciudad de Guatemala y municipios aledaños. También automatiza en la base de datos las reglas de negocio que hoy dependen del control manual.

Capa	Tecnología
Frontend	React 18 + Vite, aplicación web responsiva
Backend	API REST en Node.js + Express
Base de datos	PostgreSQL 14 o superior, servicio administrado en la nube
Seguridad	JWT por usuario y rol, contraseñas con bcrypt, HTTPS

Verificación. El script se ejecutó completo en PostgreSQL 18.3 (PGlite 0.5.8) el 18 de septiembre de 2026. Creó las 32 tablas y las 12 vistas sin errores. Las pruebas confirmaron que las reglas RF-04, RF-06 y RF-10 rechazan los datos inválidos, y que RF-05, RF-08, RF-09 y RF-16 generan su alerta o su registro en la bitácora.

Los diagramas se generaron con Mermaid a partir de fuentes editables. Todos los archivos están en la carpeta transmetro-sic\entregables.

1. Diagrama Entidad Relación

El modelo tiene 32 tablas en 6 dominios, todas en el esquema transmetro. Primero va el modelo completo y después un diagrama por dominio, que es el que se lee en tamaño carta.

Notación. Pata de gallo (crow's foot). || = exactamente uno, |o = cero o uno, o{ = cero o muchos. PK = llave primaria, FK = llave foránea, UK = llave única. En los diagramas por dominio, una entidad con solo su id y la nota "ver su dominio" pertenece a otro dominio.

Convenciones del modelo

- **Toda tabla tiene una llave sustituta id y una llave natural única: codigo, placa, dpi.**
- **Los borrados son lógicos (estado o activo), porque la bitácora exige trazabilidad permanente.**
- **Los datos derivados no se guardan: la longitud de una línea se calcula con la vista v_linea_longitud.**
- **Las asignaciones con historia (bus–línea y bus–parqueo) tienen vigencia: fecha_fin en NULL = asignación vigente.**

1.1 Modelo completo

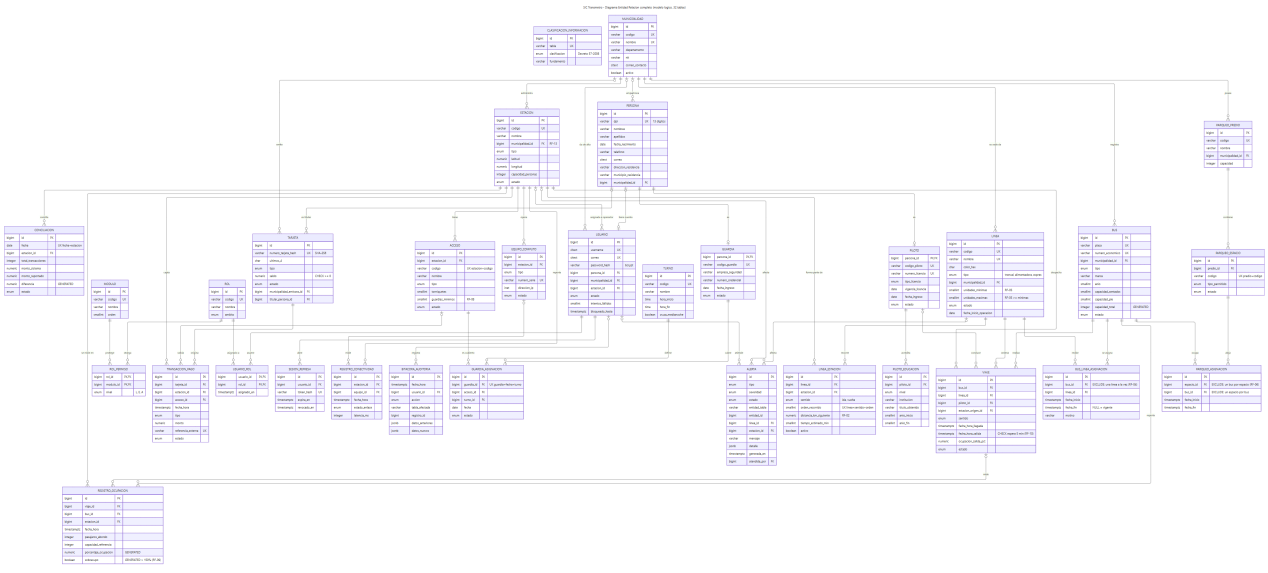


Figura 1. Diagrama entidad relación completo, 32 tablas

1.2 Dominio de red: líneas, estaciones y accesos (RF-01, RF-02, RF-03, RF-13)

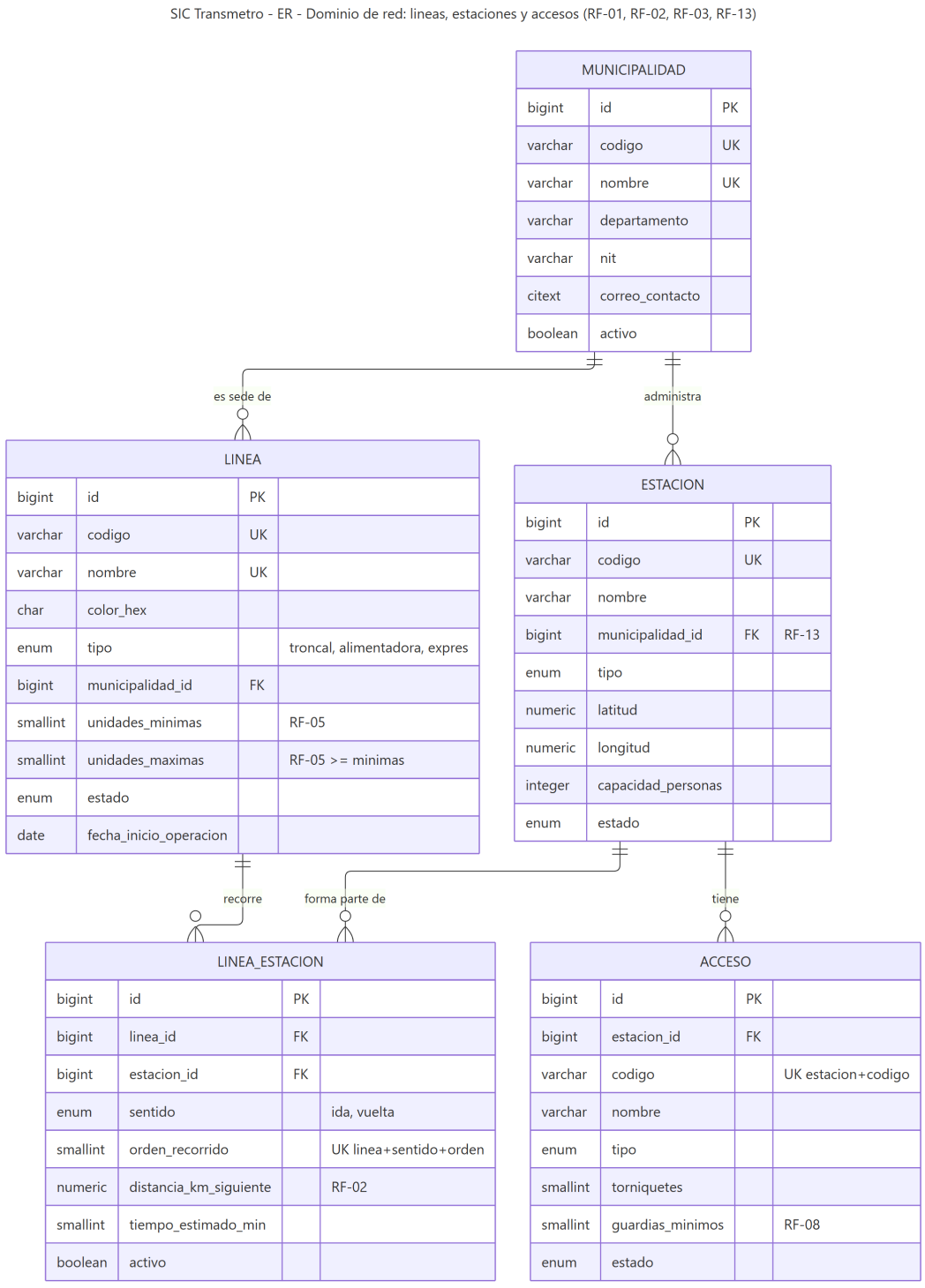


Figura 2. ER del dominio de red

LINEA y ESTACION tienen una relación muchos a muchos que se resuelve con LINEA_ESTACION. Esa tabla guarda el sentido, el orden de recorrido y la distancia a la siguiente estación. El orden no se repite dentro de la misma línea y sentido.

1.3 Dominio de flota: unidades, asignación y parqueos (RF-04, RF-05, RF-06)

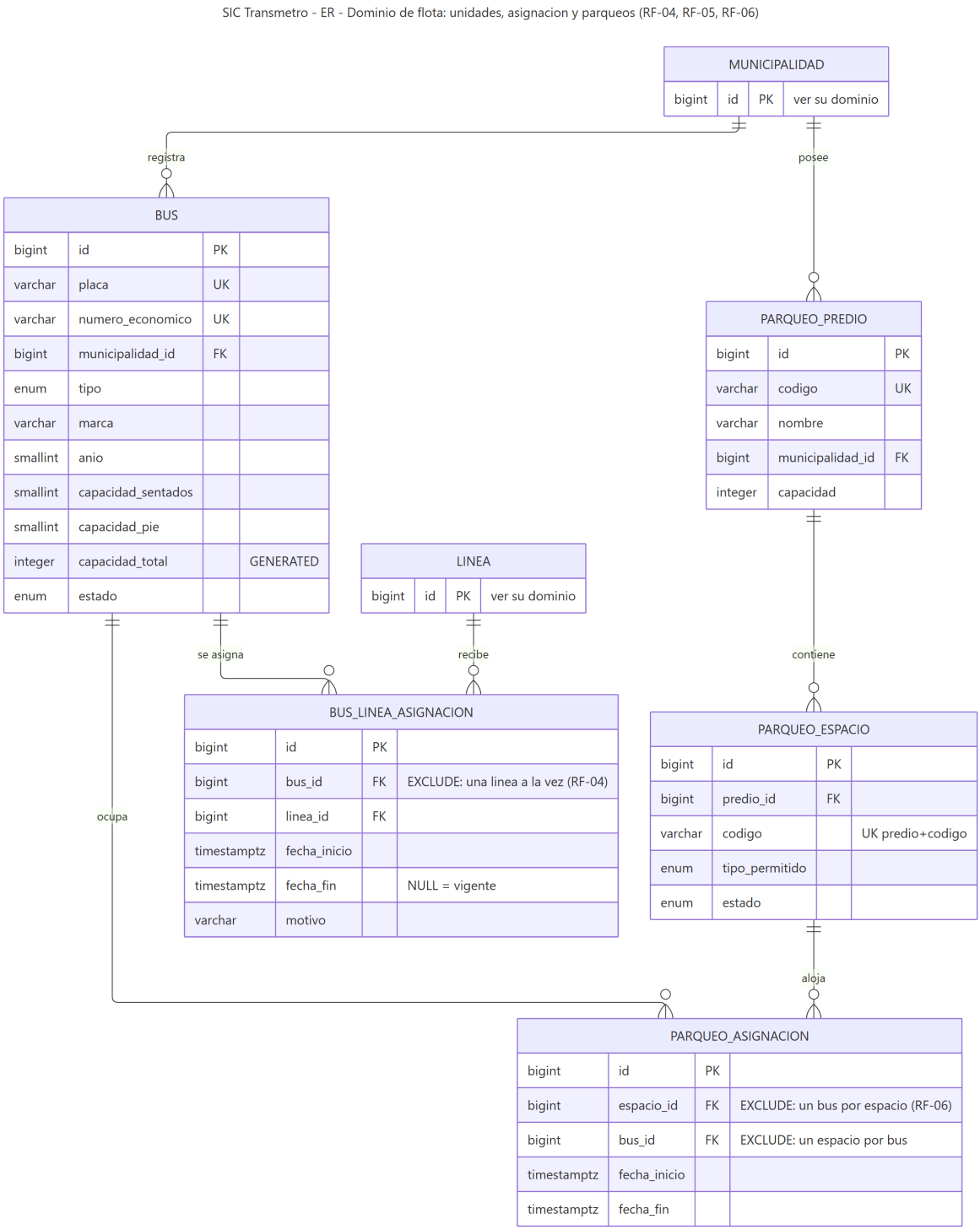


Figura 3. ER del dominio de flota

BUS_LINEA_ASIGNACION y PARQUEO_ASIGNACION guardan la historia completa. Las restricciones EXCLUDE impiden que un bus esté en dos líneas o en dos espacios a la vez, y que un espacio aloje dos buses.

1.4 Dominio de personal: pilotos, guardias y cobertura (RF-07, RF-08)

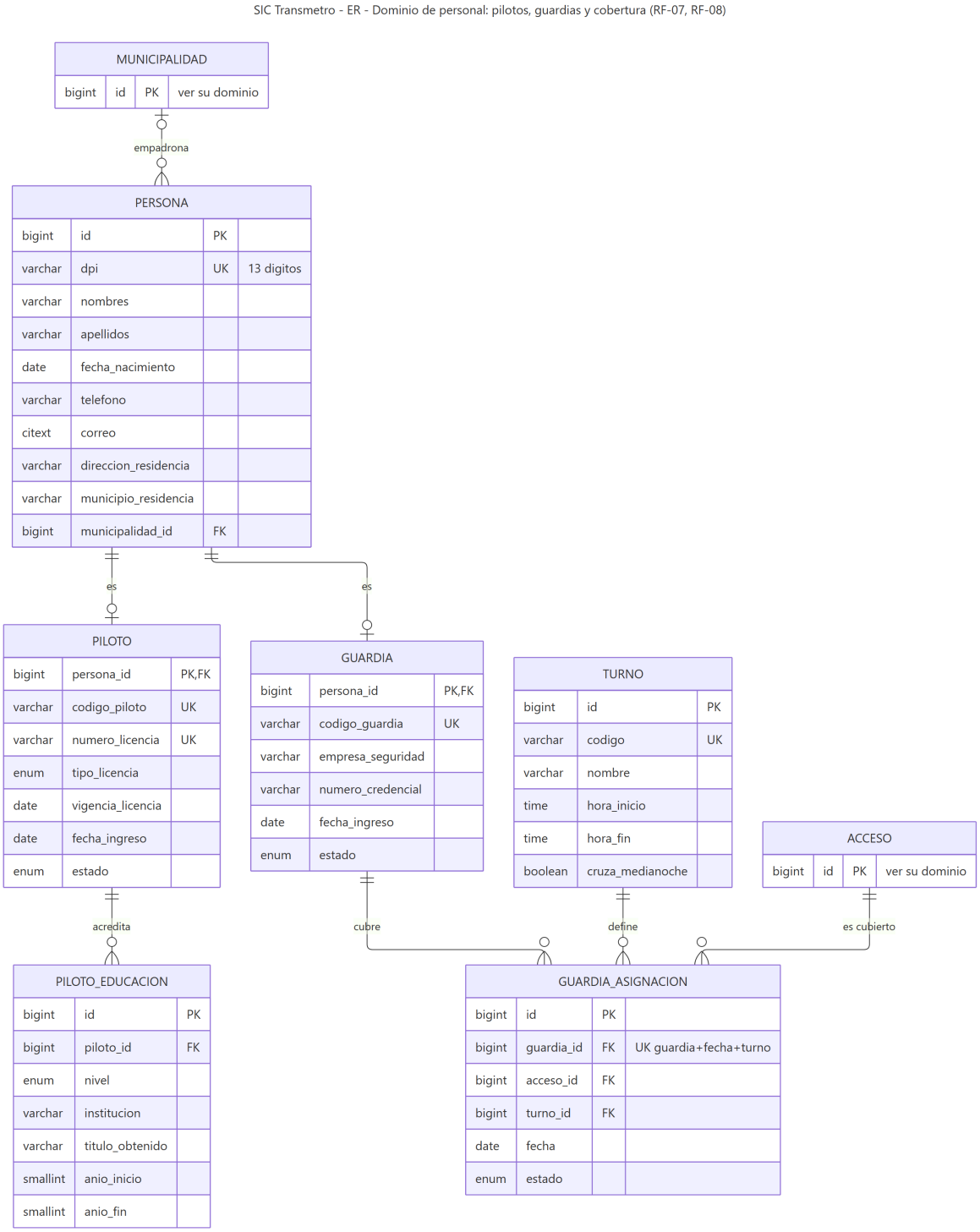


Figura 4. ER del dominio de personal

PERSONA es la generalización: PILOTO y GUARDIA usan como llave primaria el mismo persona_id (relación 1 a 0..1). GUARDIA_ASIGNACION registra la cobertura por acceso, fecha y turno.

1.5 Dominio de operación y técnica (RF-09, RF-10, RF-11)

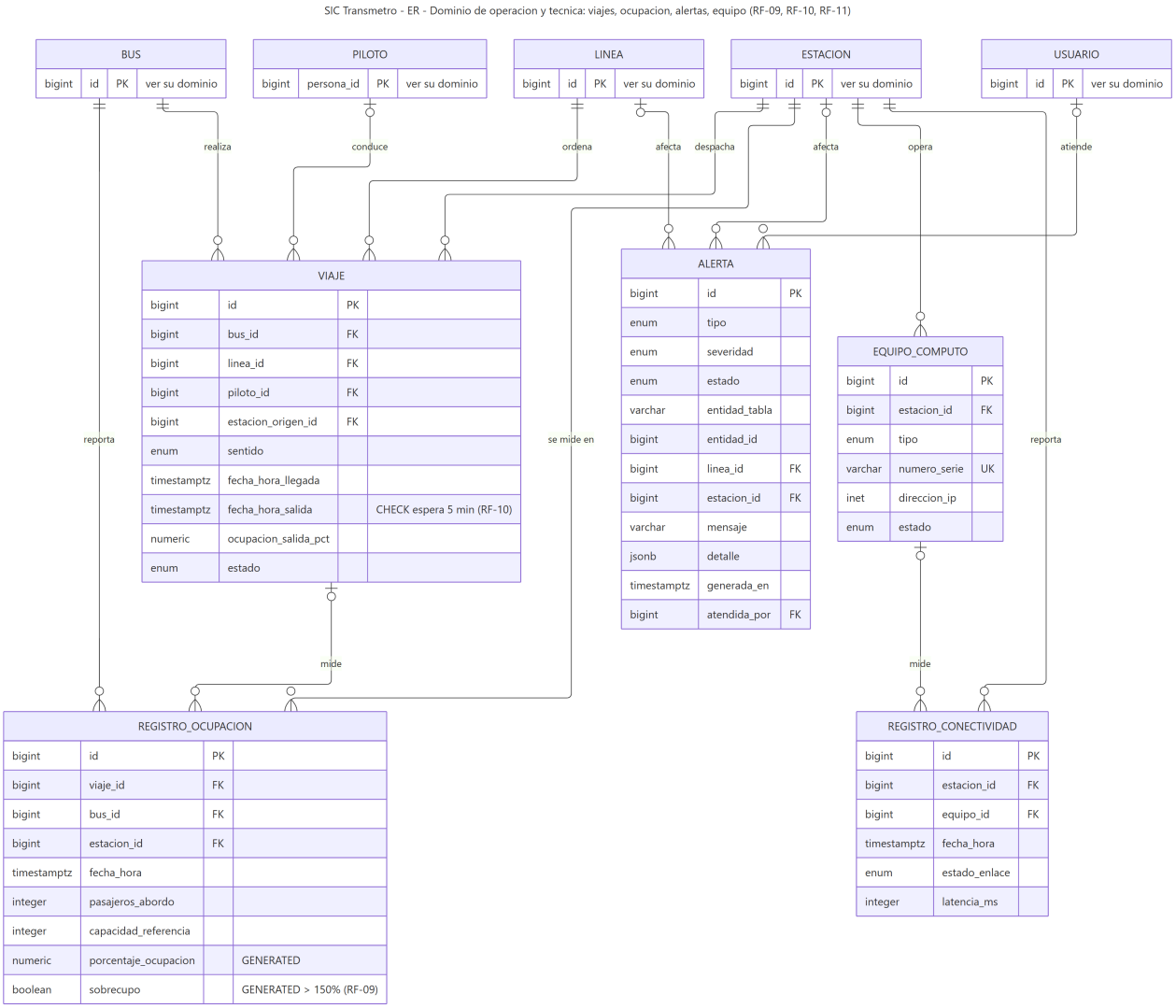


Figura 5. ER del dominio de operación

REGISTRO_OCUPACION calcula porcentaje_ocupacion y sobrecupo como columnas generadas. ALERTA es la bandeja única donde los triggers dejan cada excepción operativa.

1.6 Dominio de pagos (RF-12)

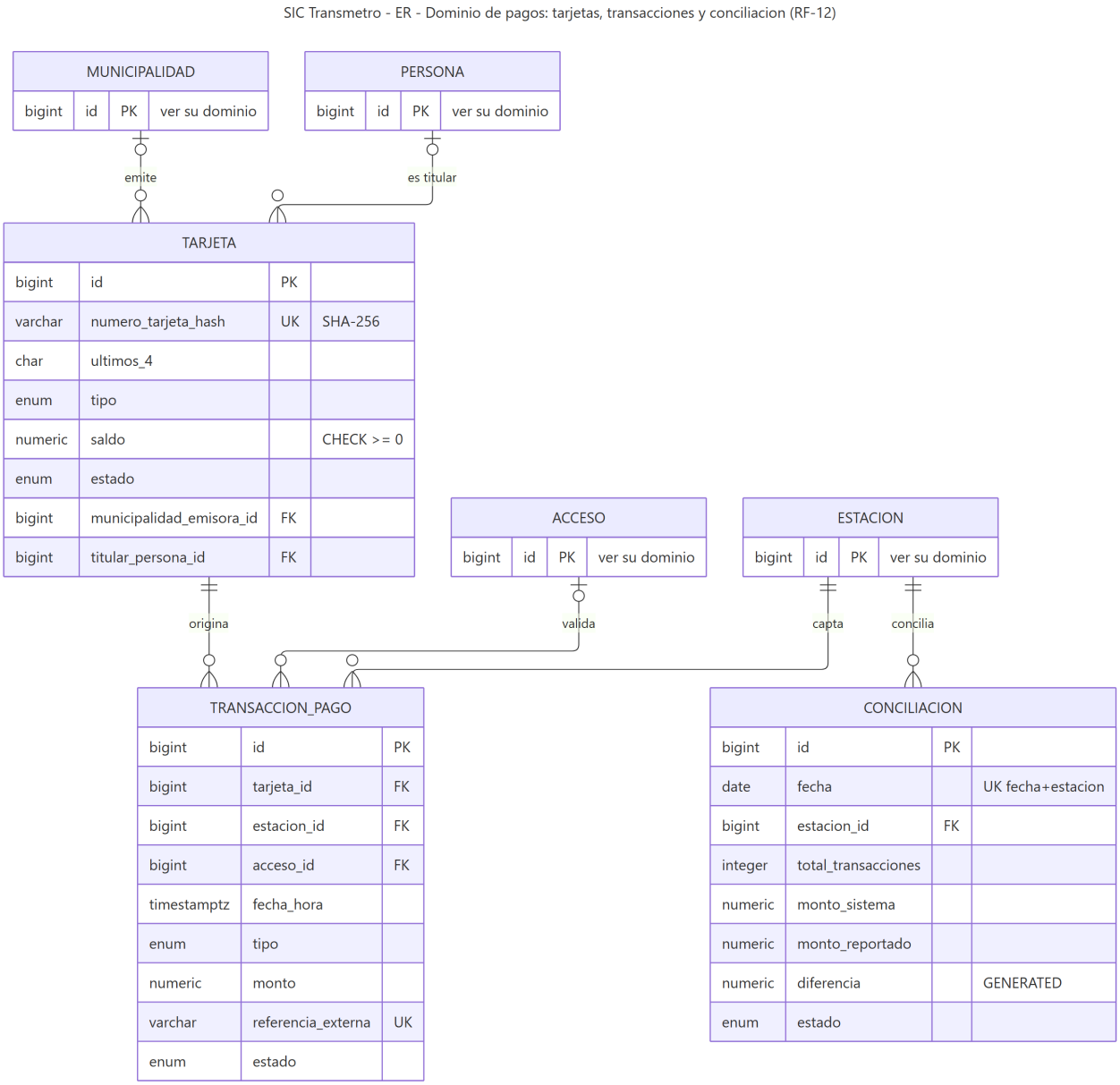


Figura 6. ER del dominio de pagos

El número de tarjeta nunca se guarda en claro: solo su hash SHA-256 y los últimos 4 dígitos. referencia_externa es única para que una transacción reenviada no se duplique.

1.7 Dominio de seguridad (RF-15, RF-16)

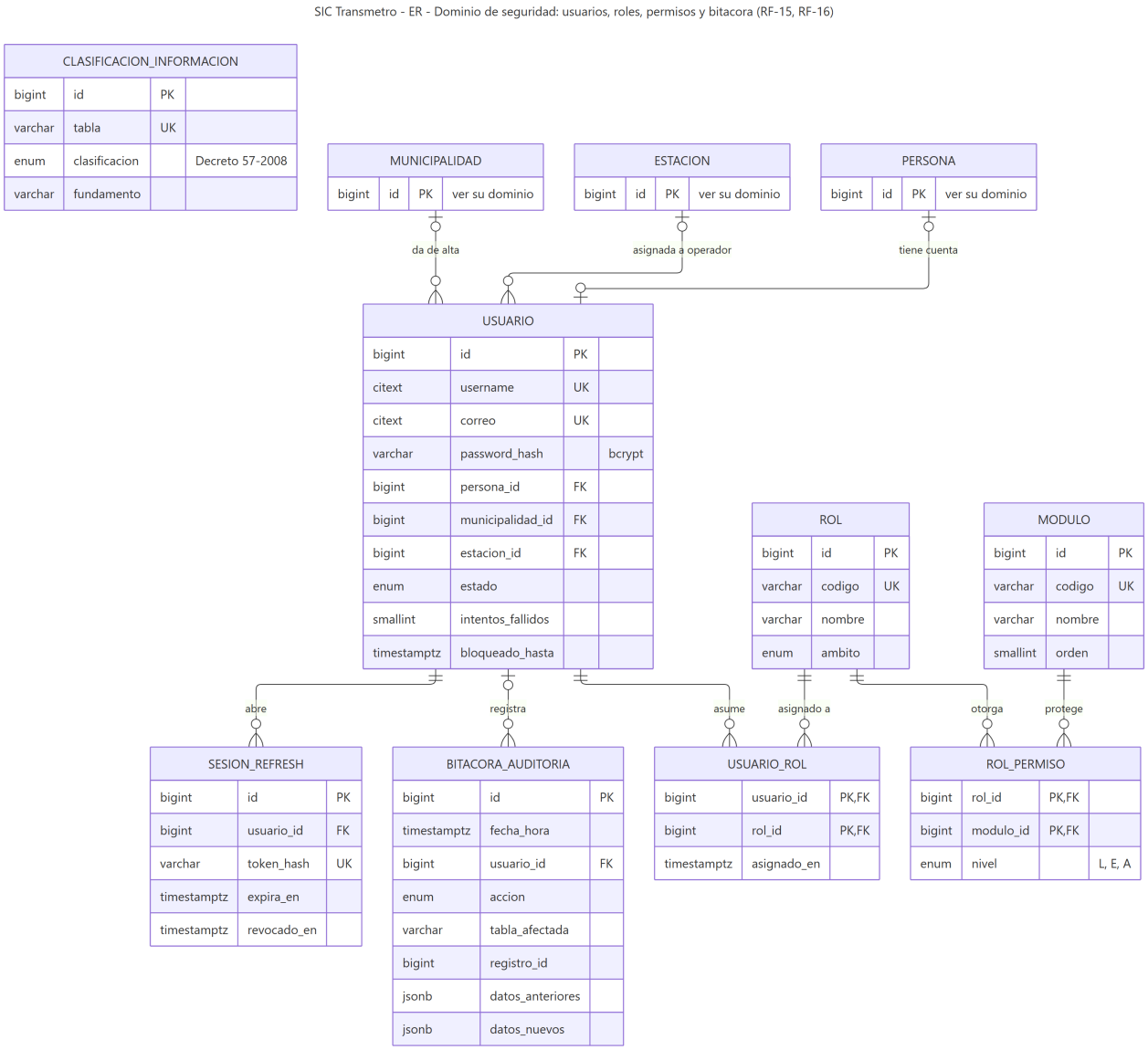


Figura 7. ER del dominio de seguridad

ROL_PERMISO es la matriz de permisos normalizada (rol × módulo × nivel L, E o A). Si no hay fila, el rol no tiene acceso a ese módulo.

1.8 Cardinalidades principales

Relación	Cardinalidad	Regla
Municipalidad – Estación	1 : N	Toda estación pertenece a una municipalidad (RF-13)

Línea – Estación	**M : N**	**Mediante LINEA_ESTACION, con orden y distancia (RF-01, RF-02)**
Estación – Acceso	**1 : N**	**Inventario de accesos (RF-03)**
Bus – Línea	**N : 1 vigente**	**Una sola asignación sin fecha_fin (RF-04)**
Bus – Espacio de parqueo	**1 : 1 vigente**	**Exclusión por periodo en ambos sentidos (RF-06)**
Acceso – Guardia	**M : N por fecha y turno**	**Mediante GUARDIA_ASIGNACION (RF-08)**
Persona – Piloto / Guardia	**1 : 0..1**	**Generalización**
Viaje – Registro de ocupación	**1 : N**	**Una medición por estación (RF-09)**
Tarjeta – Transacción	**1 : N**	**(RF-12)**
Usuario – Rol	**M : N**	**Mediante USUARIO_ROL (RF-15)**
Rol – Módulo	**M : N con nivel**	**Mediante ROL_PERMISO (RF-15)**

2. Script de base de datos

El script completo tiene 2,299 líneas y crea 32 tablas, 12 vistas, 15 funciones, 29 triggers y los datos de ejemplo. Se puede ejecutar varias veces: su primera sección elimina el esquema transmetro.

Archivo completo: transmetro-sic\entregables\02_Script_BD_completo.sql, que se entrega junto con este documento.

Cómo ejecutarlo

```
psql "postgresql://usuario:clave@host:5432/base" -v
ON_ERROR_STOP=1 -f 02_Script_BD_completo.sql
```

También se puede abrir y ejecutar completo en pgAdmin o DBeaver. La contraseña de los usuarios de ejemplo es Transmetro2026*.

2.1 Estructura del script

Sección	Contenido
00_drop_all	Elimina el esquema para poder ejecutar de nuevo
01_extensiones_dominios	Esquema, extensiones btree_gist, citext, pgcrypto y 31 tipos ENUM
02_tablas_seguridad	Municipalidad, módulo, rol, matriz de permisos, usuario, sesión, bitácora, clasificación de información
03_tablas_red	Línea, estación, recorrido (linea_estacion) y acceso
04_tablas_flota	Bus, asignación a línea, predio, espacio y asignación de parqueo
05_tablas_personal	Persona, piloto, historial educativo, guardia, turno y cobertura

06_tablas_operacion	Alerta, viaje, registro de ocupación, equipo de cómputo y conectividad
07_tablas_pagos	Tarjeta, transacción de pago y conciliación
08_reglas_negocio	Funciones y triggers de reglas de negocio y auditoría
09_indices	37 índices para el tiempo de respuesta menor a 3 segundos
10_vistas	12 vistas de consulta y reportes
11_seed	Catálogos obligatorios y datos de ejemplo

2.2 Reglas de negocio dentro de la base

Las reglas viven en PostgreSQL, no en la API. Así ningún cliente puede saltárselas: ni la aplicación web, ni un script de carga, ni una consulta manual.

Requerimiento	Mecanismo
RF-04 Un bus en una sola línea activa	EXCLUDE USING gist sobre bus y periodo + índice único parcial
RF-05 Rango mínimo y máximo de unidades	Trigger que abre o cierra la alerta rango_unidades
RF-06 Ningún bus sin parqueo	Dos EXCLUDE + trigger de alerta bus_sin_parqueo
RF-08 Cobertura mínima de guardias	Trigger que compara contra acceso.guardias_minimos
RF-09 Sobrecupo del 50 % adicional	Columnas GENERATED + trigger de alerta
RF-10 Espera de 5 minutos	CHECK sobre la fila del viaje
RF-16 Bitácora de auditoría	Trigger genérico fn_auditoria() sobre 14 tablas

RF-04 — un bus solo puede estar activo en una línea a la vez

```
CREATE TABLE bus_linea_asignacion (
   id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   bus_id      bigint      NOT NULL REFERENCES bus(id)   ON DELETE
CASCADE,
   linea_id    bigint      NOT NULL REFERENCES linea(id) ON DELETE
RESTRICT,
   fecha_inicio timestamptz NOT NULL DEFAULT now(),
   fecha_fin    timestamptz,
   motivo       varchar(250),
   CONSTRAINT ck_bus_asig_fechas CHECK (fecha_fin IS NULL OR
fecha_fin > fecha_inicio),
   CONSTRAINT ex_bus_una_linea_a_la_vez EXCLUDE USING gist (
      bus_id                              WITH =,
      tstzrange(fecha_inicio, fecha_fin, '[)') WITH &&
   )
);

CREATE UNIQUE INDEX ux_bus_asignacion_vigente
   ON bus_linea_asignacion (bus_id)
   WHERE fecha_fin IS NULL;
```

RF-09 — ocupación y sobrecupo calculados por la base

```
porcentaje_ocupacion numeric(6,2) GENERATED ALWAYS AS (
   round(pasajeros_abordo * 100.0 / capacidad_referencia, 2)
) STORED,

sobrecupo boolean GENERATED ALWAYS AS (
   pasajeros_abordo > capacidad_referencia * 1.5
) STORED,
```

RF-10 — espera mínima de 5 minutos con menos del 25 % de ocupación

```
CONSTRAINT ck_viaje_espera_minima CHECK (
    fecha_hora_salida IS NULL
    OR ocupacion_salida_pct IS NULL
    OR ocupacion_salida_pct >= 25
    OR (fecha_hora_salida - fecha_hora_llegada) >= INTERVAL '5
minutes'
)
```

RF-05 — alerta automática por rango de unidades (extracto)

```
IF v_activas < v_linea.unidades_minimas THEN
    PERFORM fn_alerta_abrir(
        'rango_unidades', 'critica', 'linea', p_linea_id, p_linea_id, NULL,
        format('La linea %s opera con %s unidades, por debajo del
minimo de %s.',
               v_linea.codigo, v_activas, v_linea.unidades_minimas));
ELSIF v_activas > v_linea.unidades_maximas THEN
    PERFORM fn_alerta_abrir('rango_unidades', 'advertencia', ...);
ELSE
    PERFORM fn_alerta_cerrar('rango_unidades', 'linea', p_linea_id);
END IF;
```

2.3 Resultado de la ejecución de prueba

El script se ejecutó dos veces seguidas en PostgreSQL 18.3 (PGlite 0.5.8) el 18 de septiembre de 2026, sin errores. Después se probaron las reglas con datos válidos e inválidos.

Regla	Prueba	Resultado
RF-04	Asignar a L2 un bus que ya está en L1	Rechazado por ex_bus_una_linea_a_la_vez
RF-06	Asignar un segundo bus a un espacio ocupado	Rechazado por ex_espacio_un_bus
RF-10	Despachar con 20 % de ocupación a los 3 minutos	Rechazado por ck_viaje_espera_minima

RF-10	**Despachar con 20 % de ocupación a los 6 minutos**	**Aceptado**
RF-09	**250 pasajeros en una unidad de capacidad 160**	**Alerta crítica de sobrecupo (156.25 %)**
RF-05	**Línea L1 con 12 unidades y mínimo de 28**	**Alerta crítica de rango**
RF-08	**Acceso con 2 guardias mínimos sin cobertura completa**	**Alerta de cobertura**
RF-16	**Modificar el nombre de una línea**	**Registro en bitácora con usuario, acción y campos cambiados**
RF-02	**Longitud de la L1 en ambos sentidos**	**8.380 km**

3. Diagramas UML

Son 15 diagramas de 7 tipos, que cubren la vista funcional, la estática, la dinámica y la física del sistema.

Tipo	Diagramas	Qué muestra
Casos de uso	6 (general + uno por actor)	Qué hace cada rol en el sistema
Clases	1	Modelo de dominio: clases, atributos, métodos y asociaciones
Secuencia	3	Interacción entre interfaz, API y base en tres flujos críticos
Actividades	1	Flujo de llegada, ocupación y despacho de una unidad
Estados	2	Ciclo de vida de la unidad y de la alerta
Componentes	1	Módulos del frontend, la API y la base de datos
Despliegue	1	Nodos físicos y protocolos en la nube

3.1 Diagramas de casos de uso

Hay 5 actores, uno por rol de la matriz de permisos, y 24 casos de uso. El Administrador general hereda todos los casos del Administrador municipal (generalización). En naranja van los casos incluidos o extendidos, que el sistema ejecuta de forma automática.

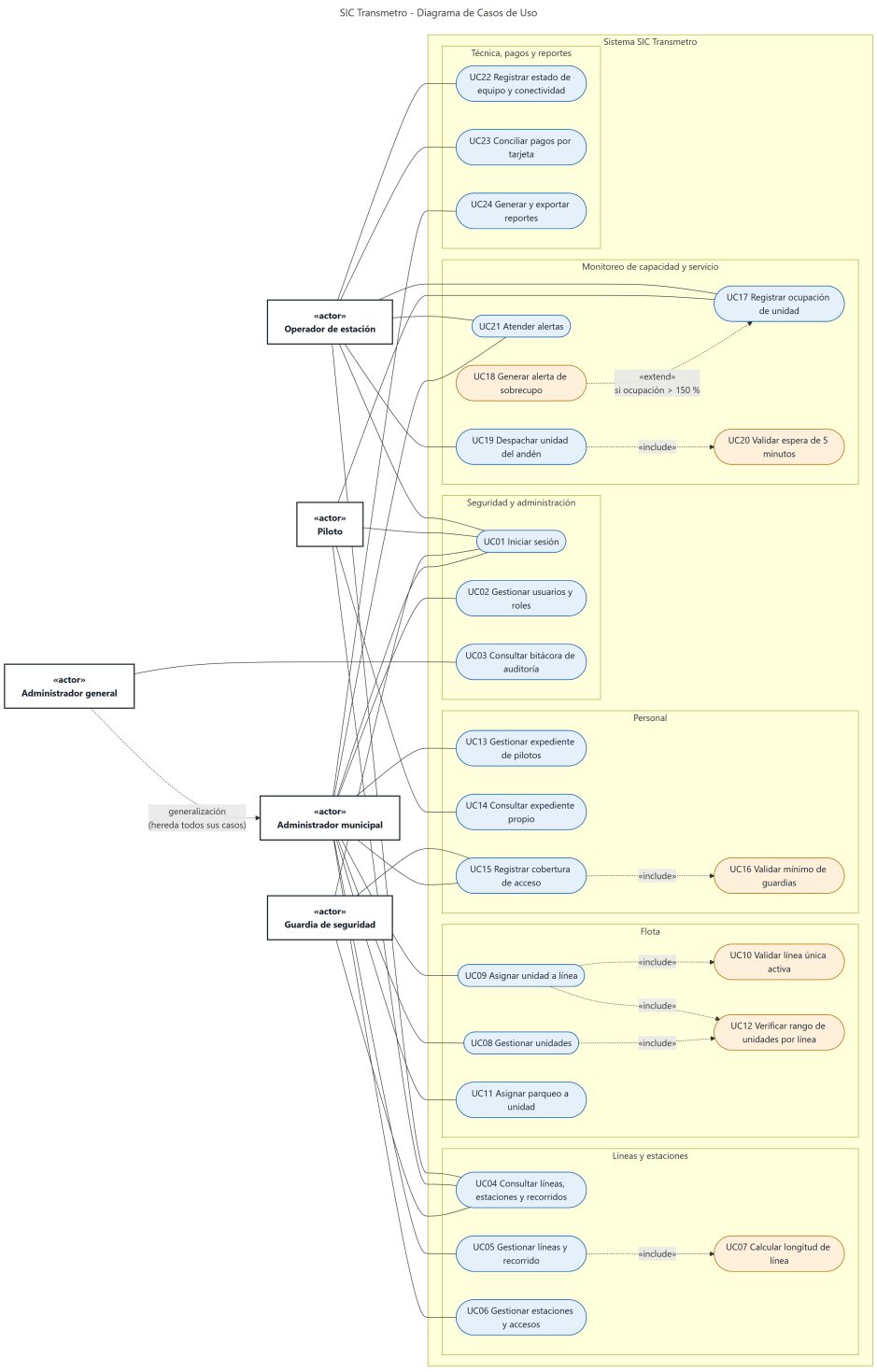


Figura 8. Casos de uso general

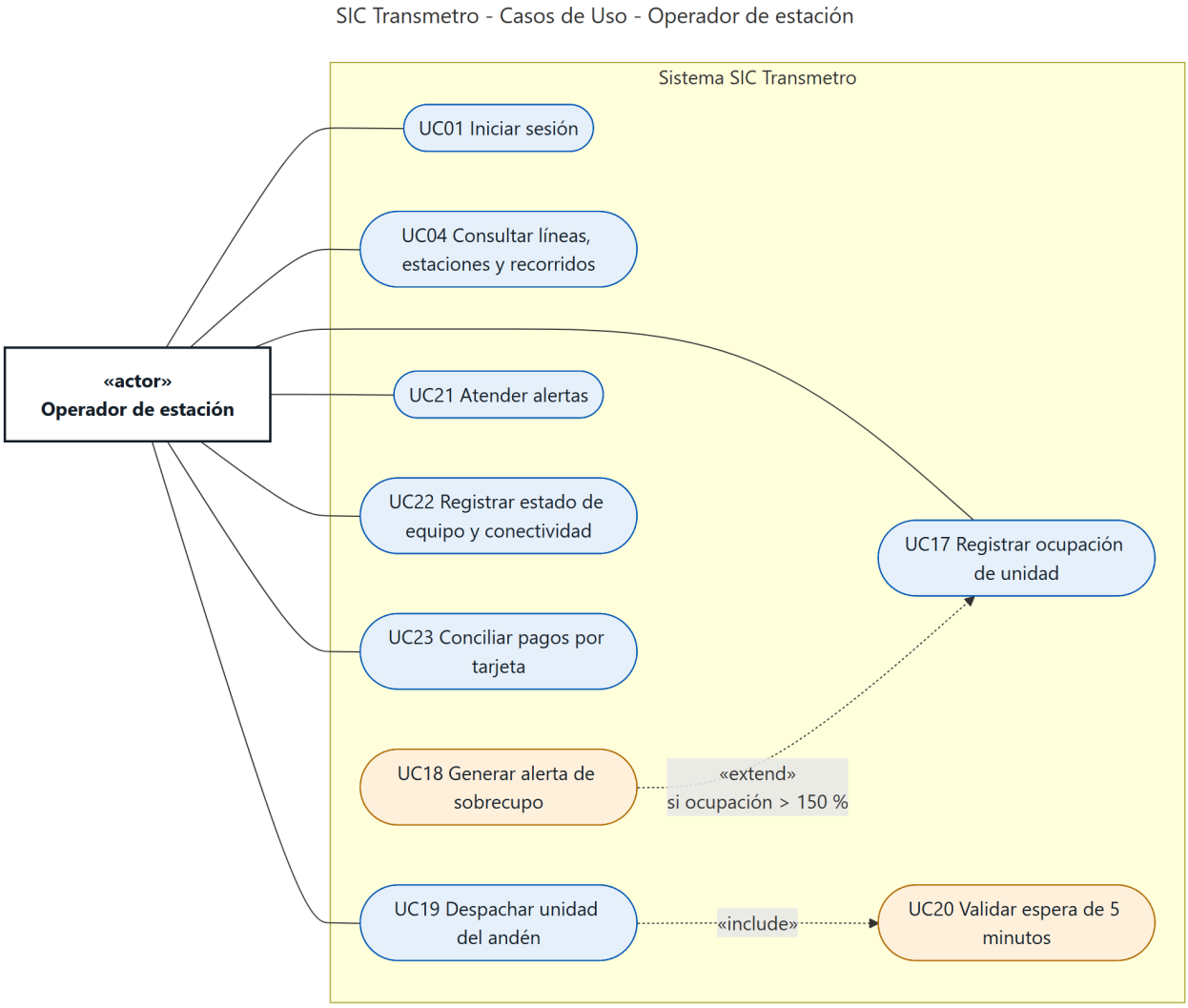


Figura 9. Casos de uso del operador de estación

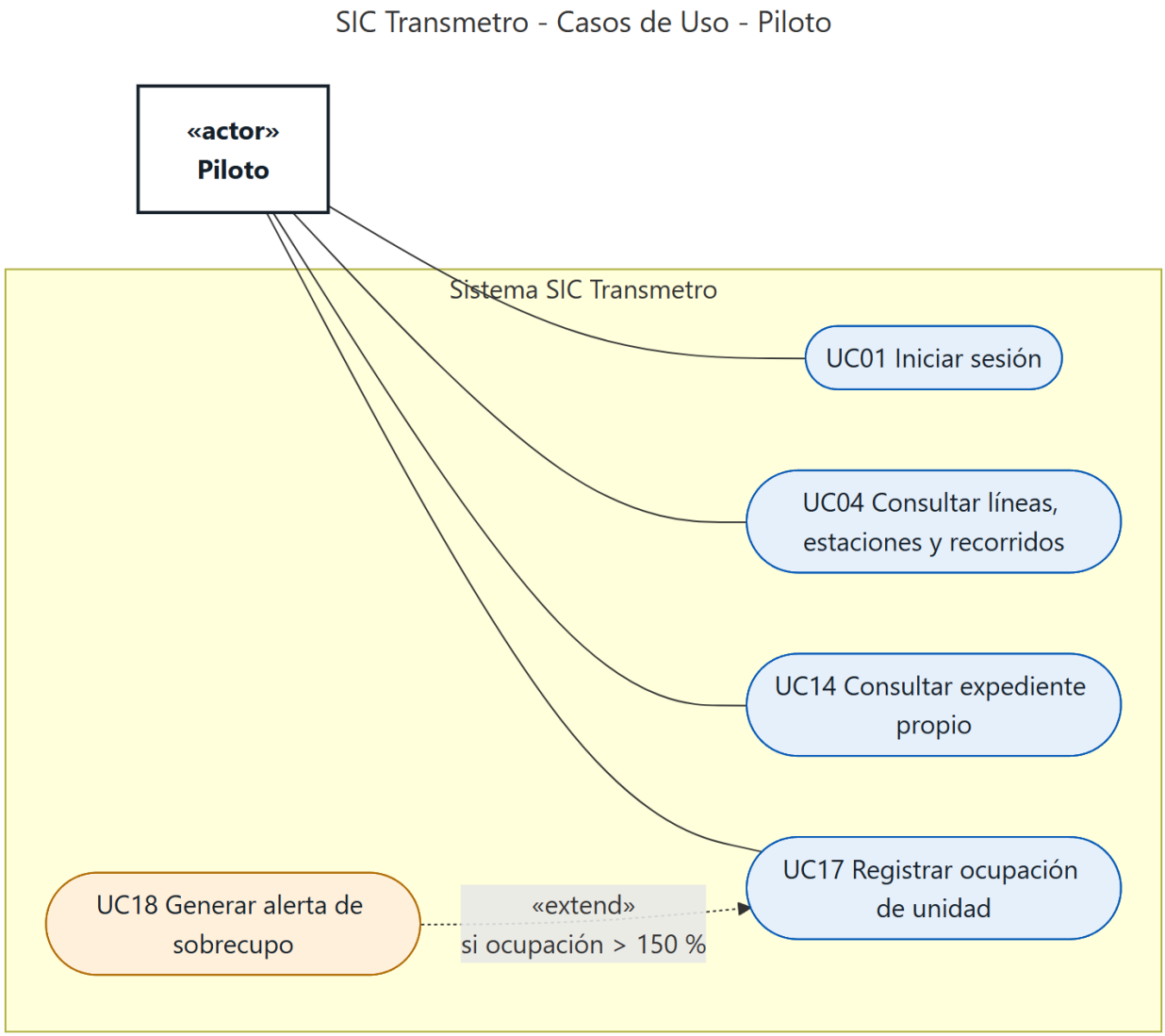


Figura 10. Casos de uso del piloto

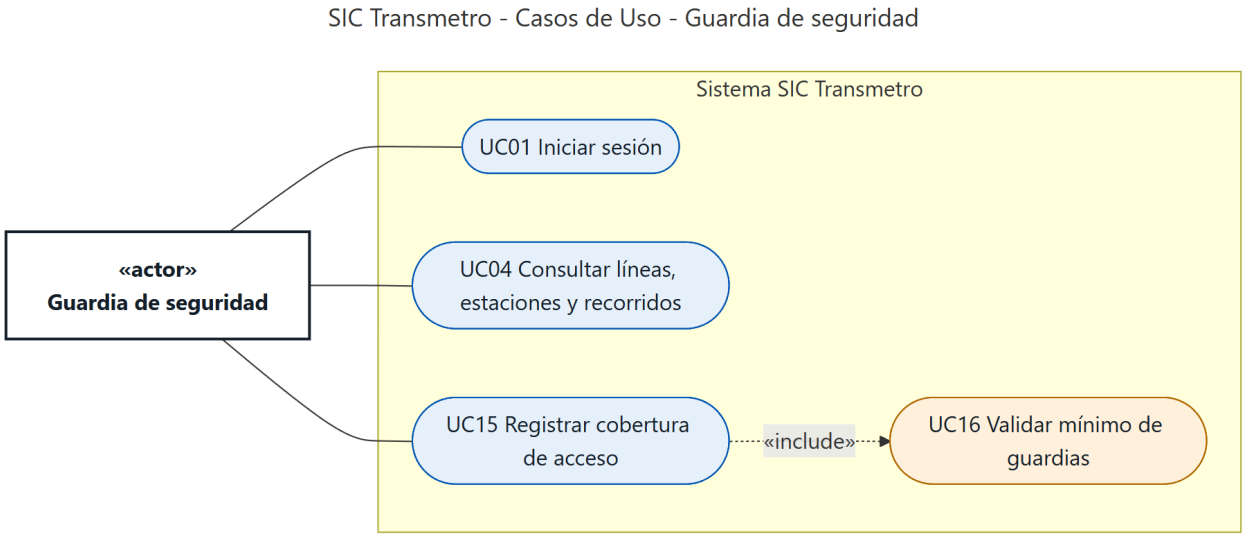


Figura 11. Casos de uso del guardia de seguridad

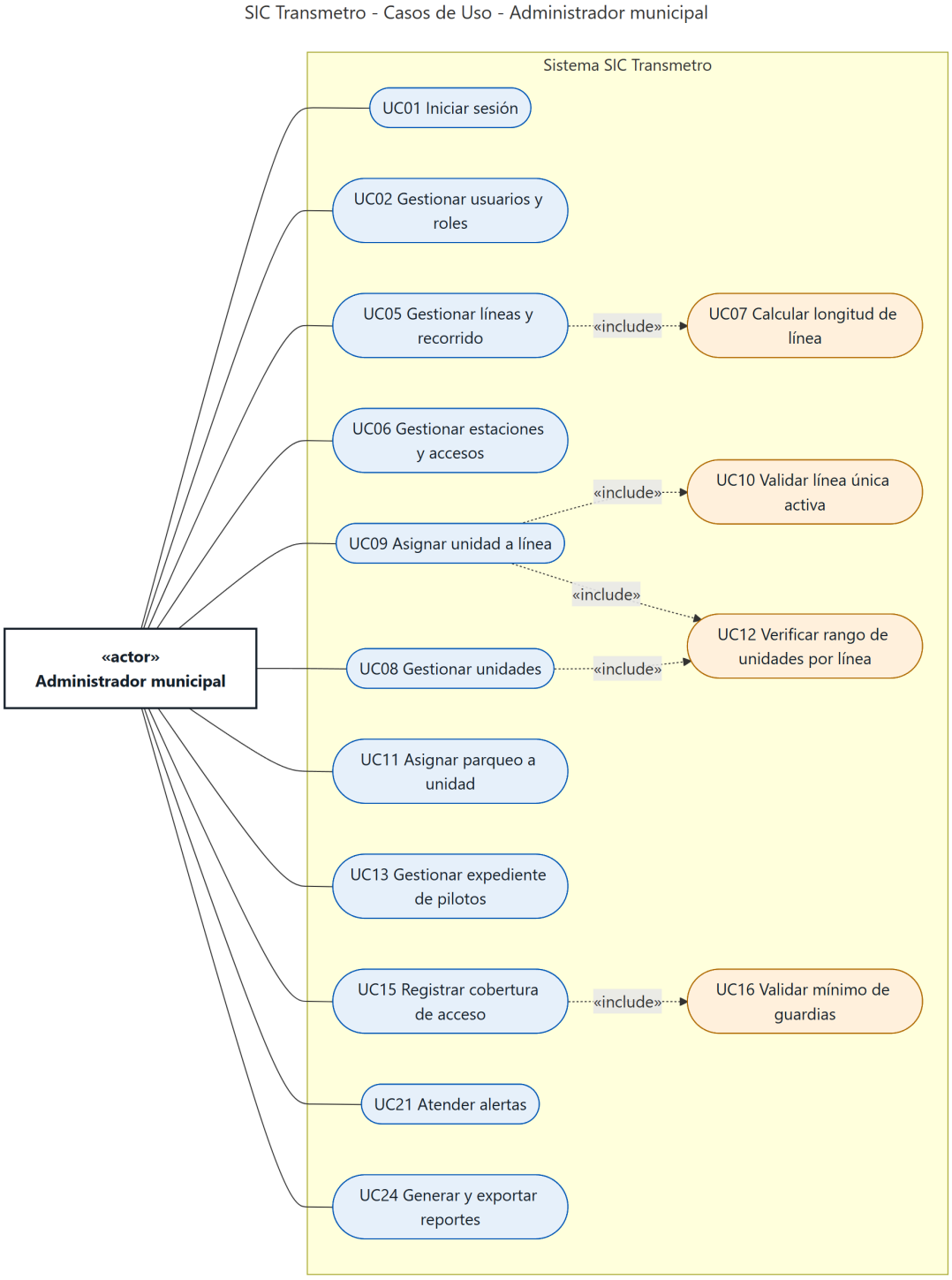


Figura 12. Casos de uso del administrador municipal

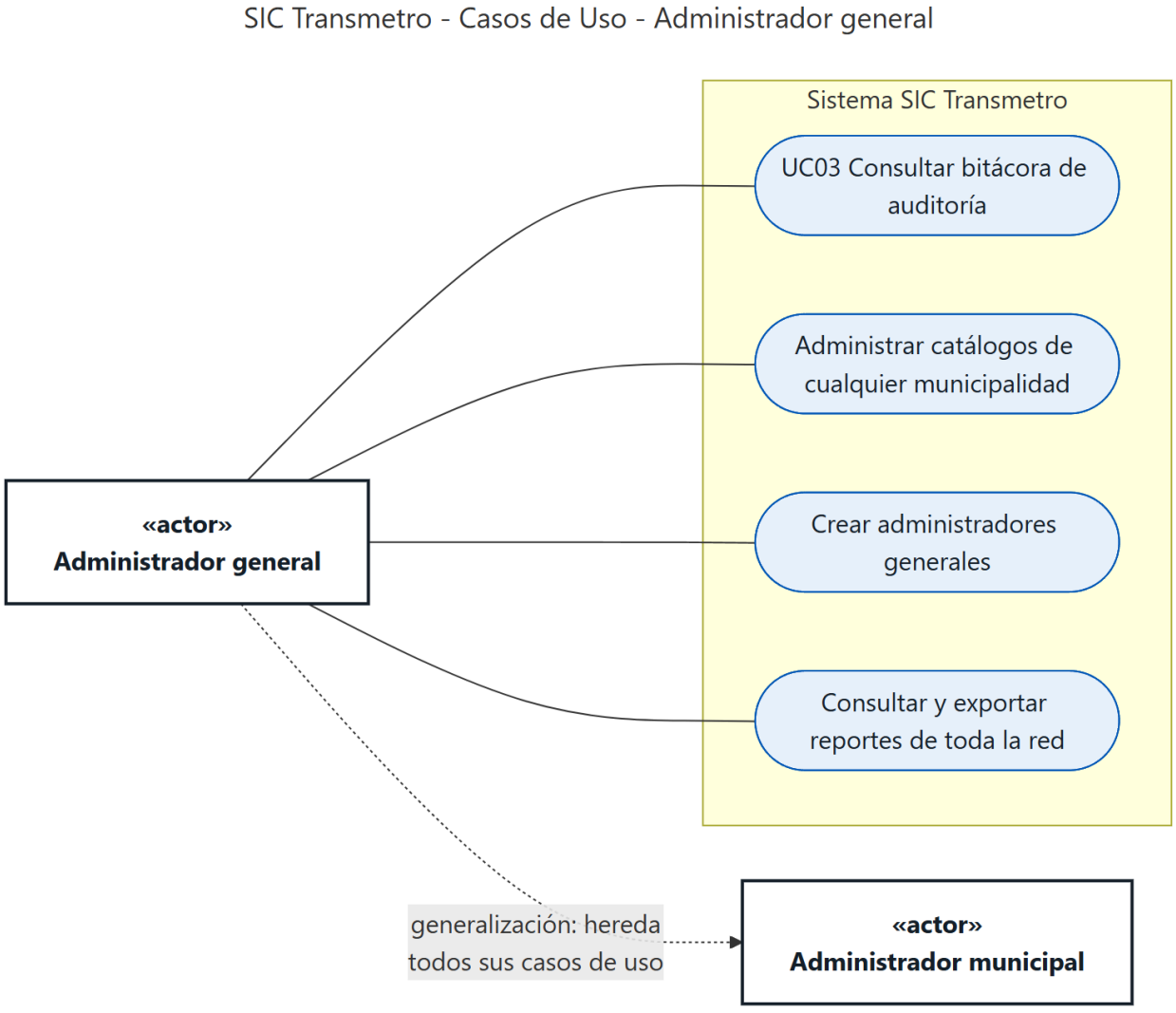


Figura 13. Casos de uso del administrador general

3.2 Diagrama de clases

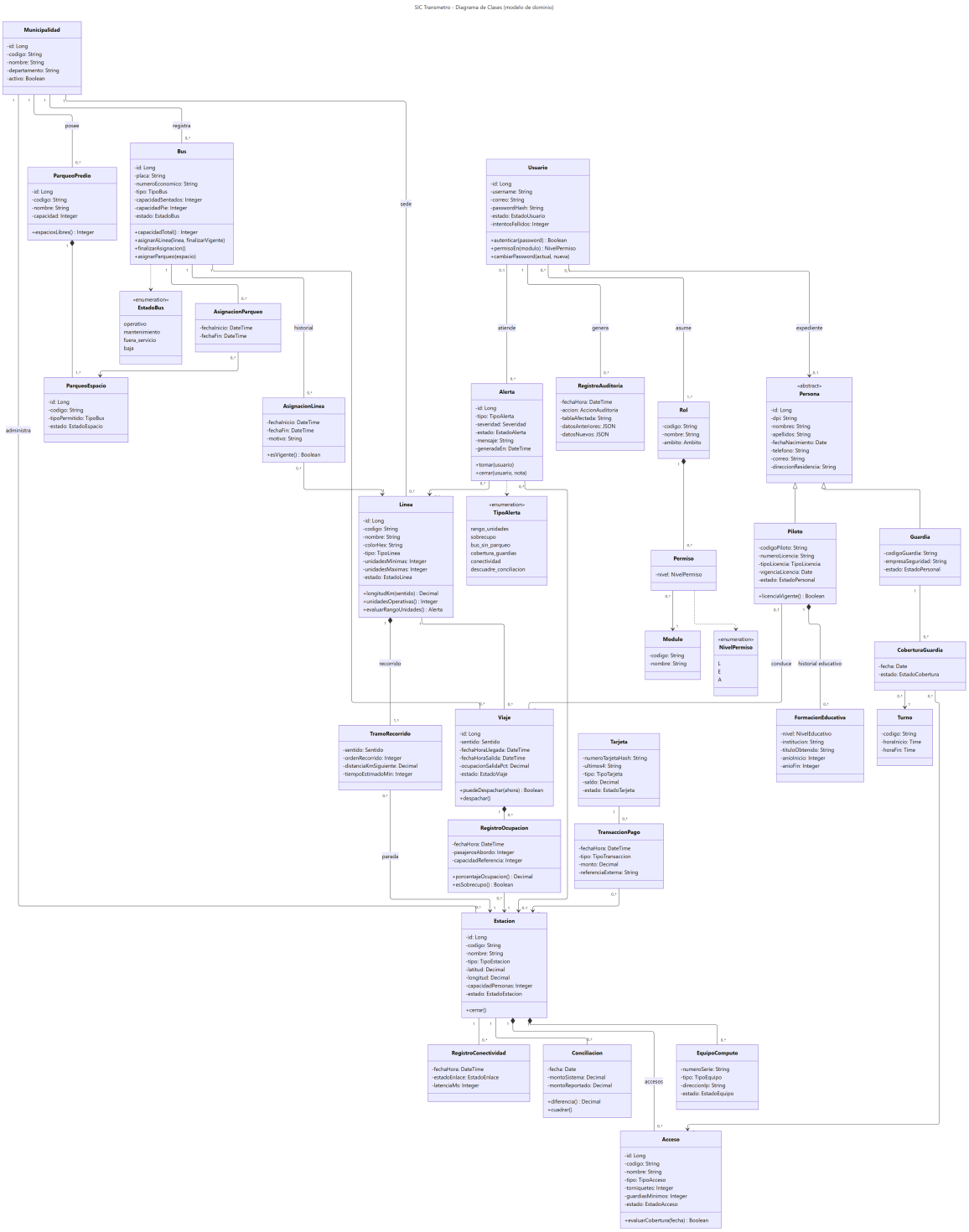


Figura 14. Diagrama de clases del modelo de dominio

Persona es abstracta y se especializa en Piloto y Guardia. Las composiciones (rombo relleno) marcan partes que no existen sin su todo: el recorrido de una línea, los accesos de una estación y los espacios de un predio. Los métodos reflejan las reglas: Bus.asignarALinea(), Viaje.puedeDespachar() y RegistroOcupacion.esSobrecupo().

3.3 Diagramas de secuencia

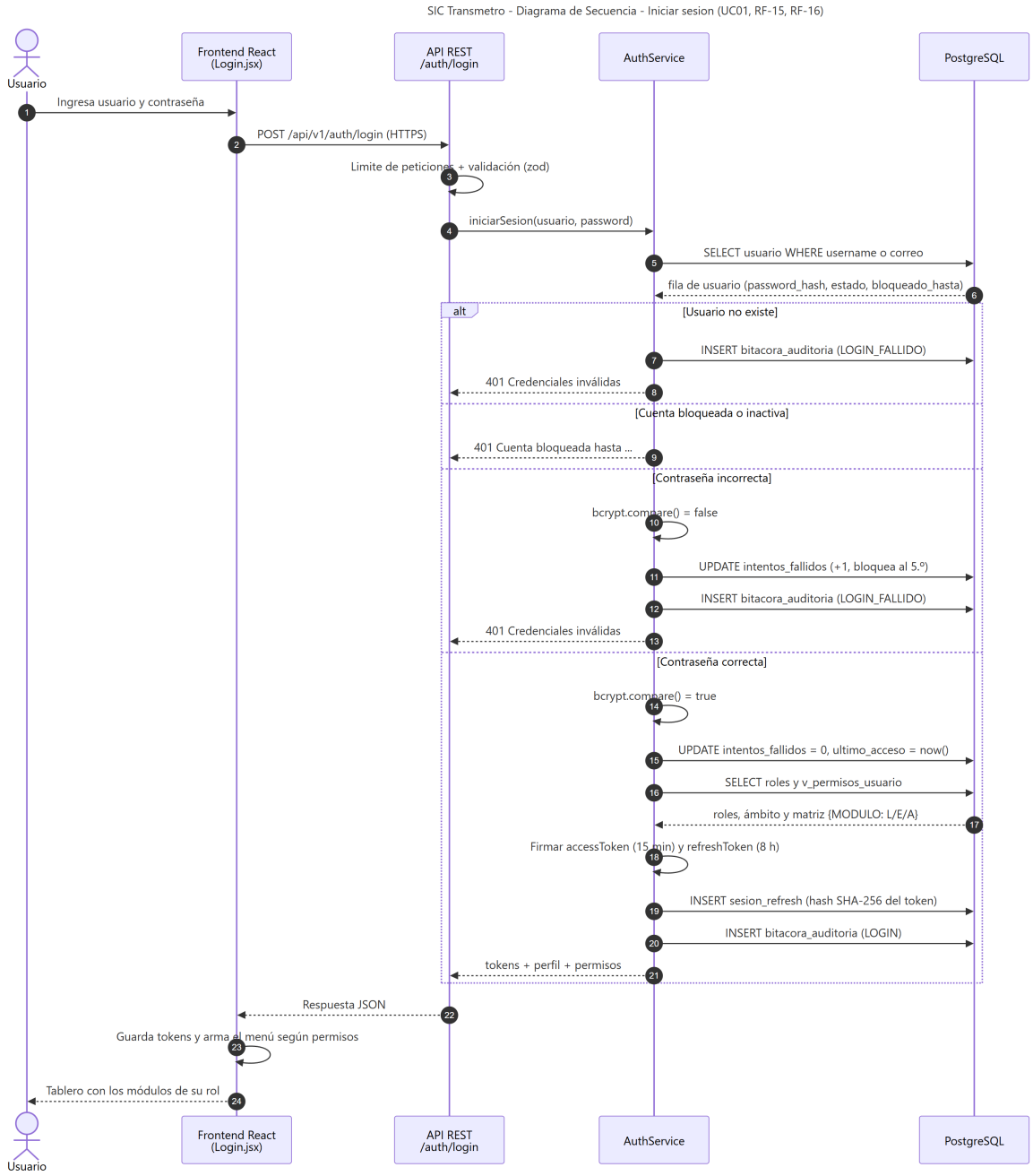


Figura 15. Secuencia de inicio de sesión (UC01, RF-15, RF-16)

El sistema bloquea la cuenta 15 minutos después de 5 intentos fallidos. Cada intento, exitoso o no, queda en la bitácora. El token de acceso lleva los permisos del usuario, con los que la interfaz arma el menú.

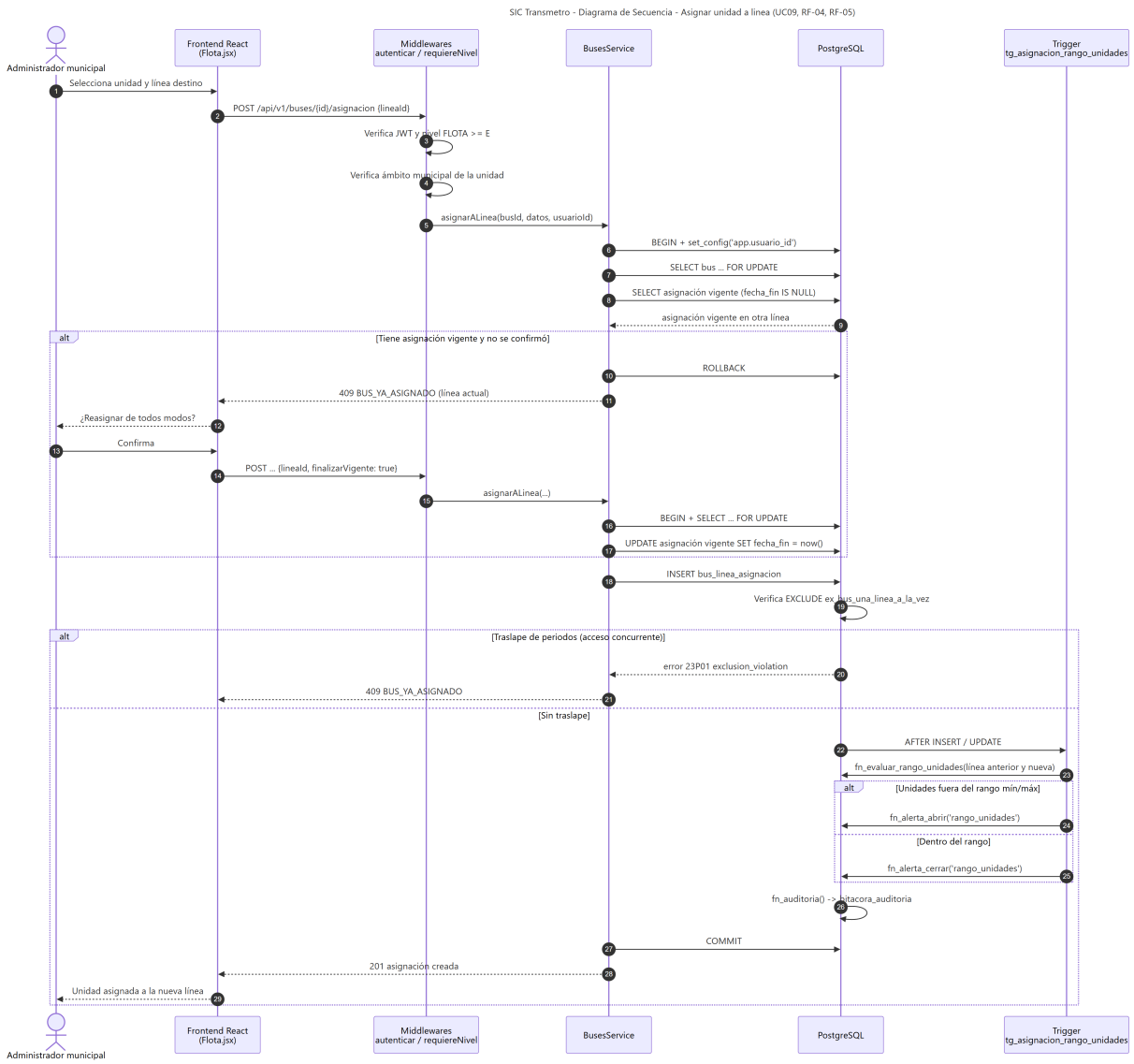


Figura 16. Secuencia de asignación de unidad a línea (UC09, RF-04, RF-05)

La regla se valida en tres capas: la interfaz pide confirmar la reasignación, la API bloquea la fila con FOR UPDATE y la base rechaza cualquier traslape con EXCLUDE. Si dos personas asignan el mismo bus al mismo tiempo, una de las dos operaciones falla.

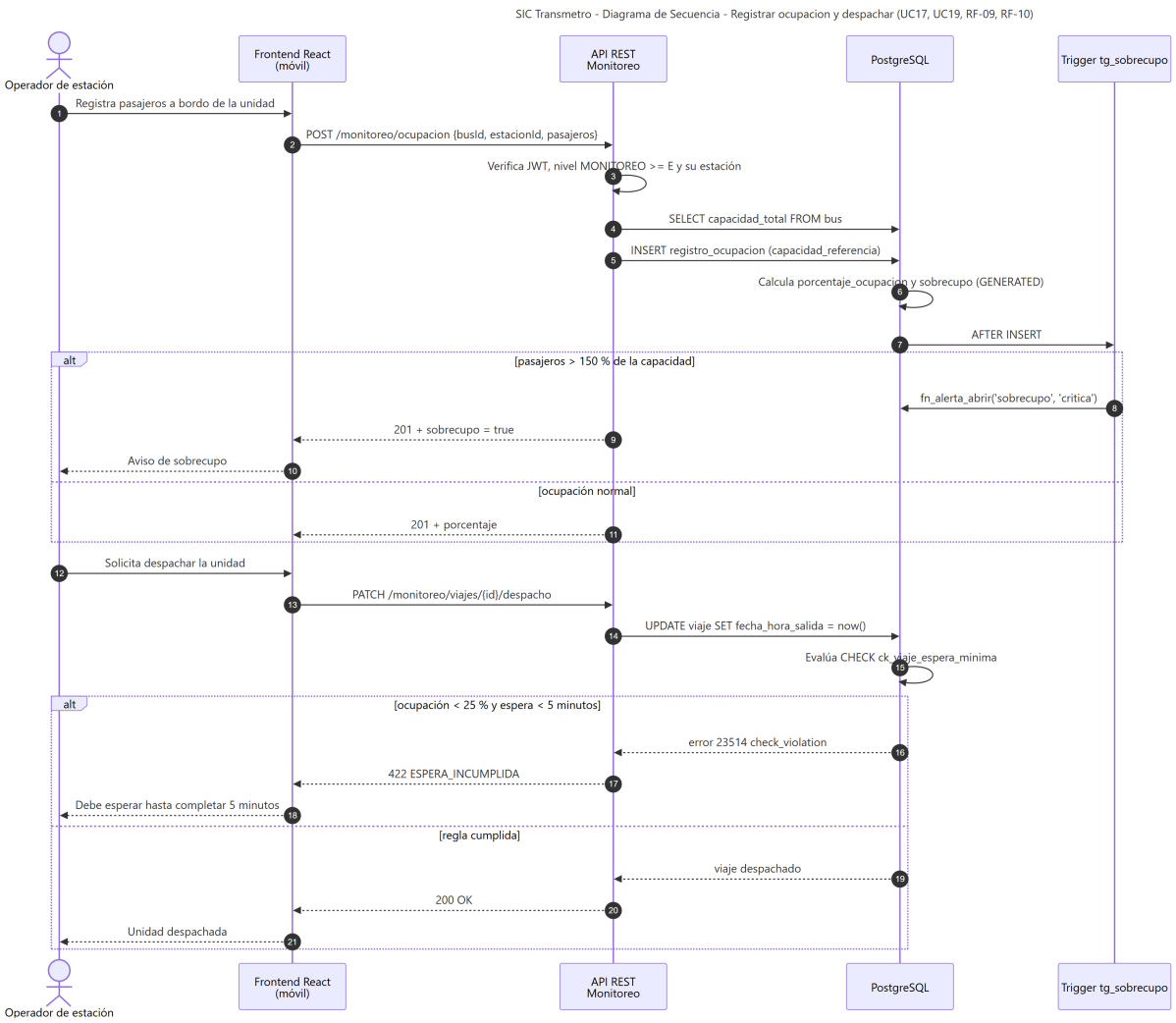


Figura 17. Secuencia de ocupación y despacho (UC17, UC19, RF-09, RF-10)

Este flujo corresponde al módulo de monitoreo, planificado para el Sprint 5. Las reglas de la base que usa (sobrecupo y espera de 5 minutos) ya están implementadas y probadas.

3.4 Diagrama de actividades

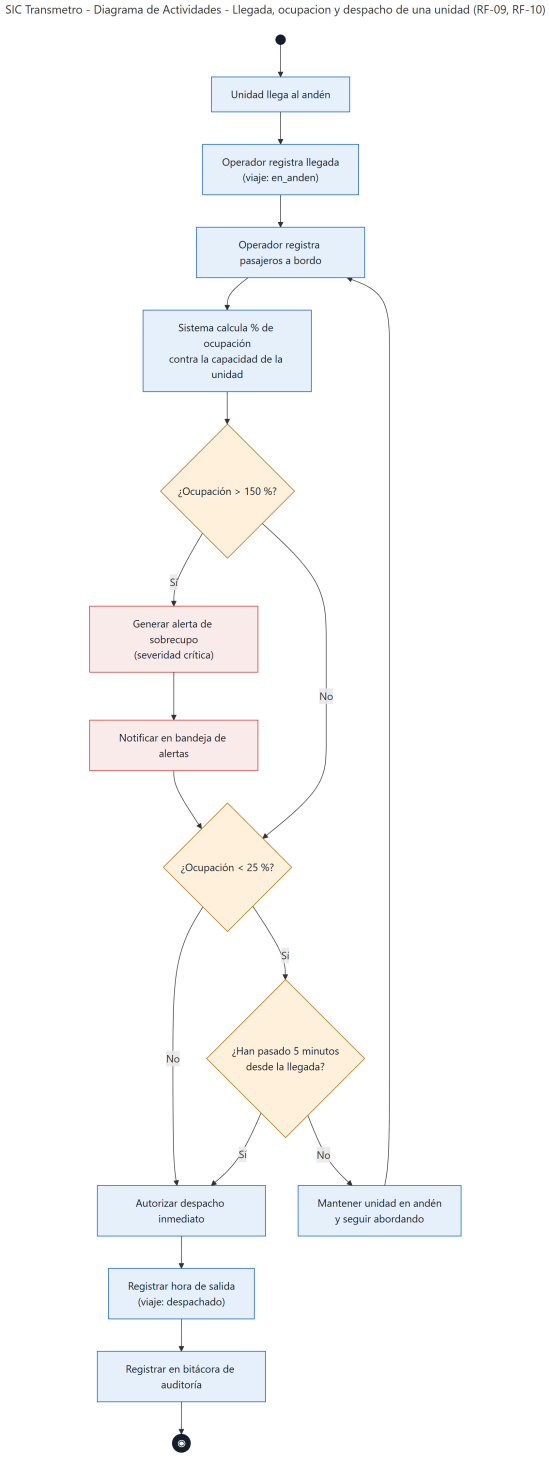


Figura 18. Actividades del despacho de una unidad (RF-09, RF-10)

Si la ocupación pasa del 150 % se genera una alerta, pero la unidad sí puede salir. Si es menor al 25 %, la unidad espera en el andén hasta completar 5 minutos desde su llegada.

3.5 Diagramas de estados

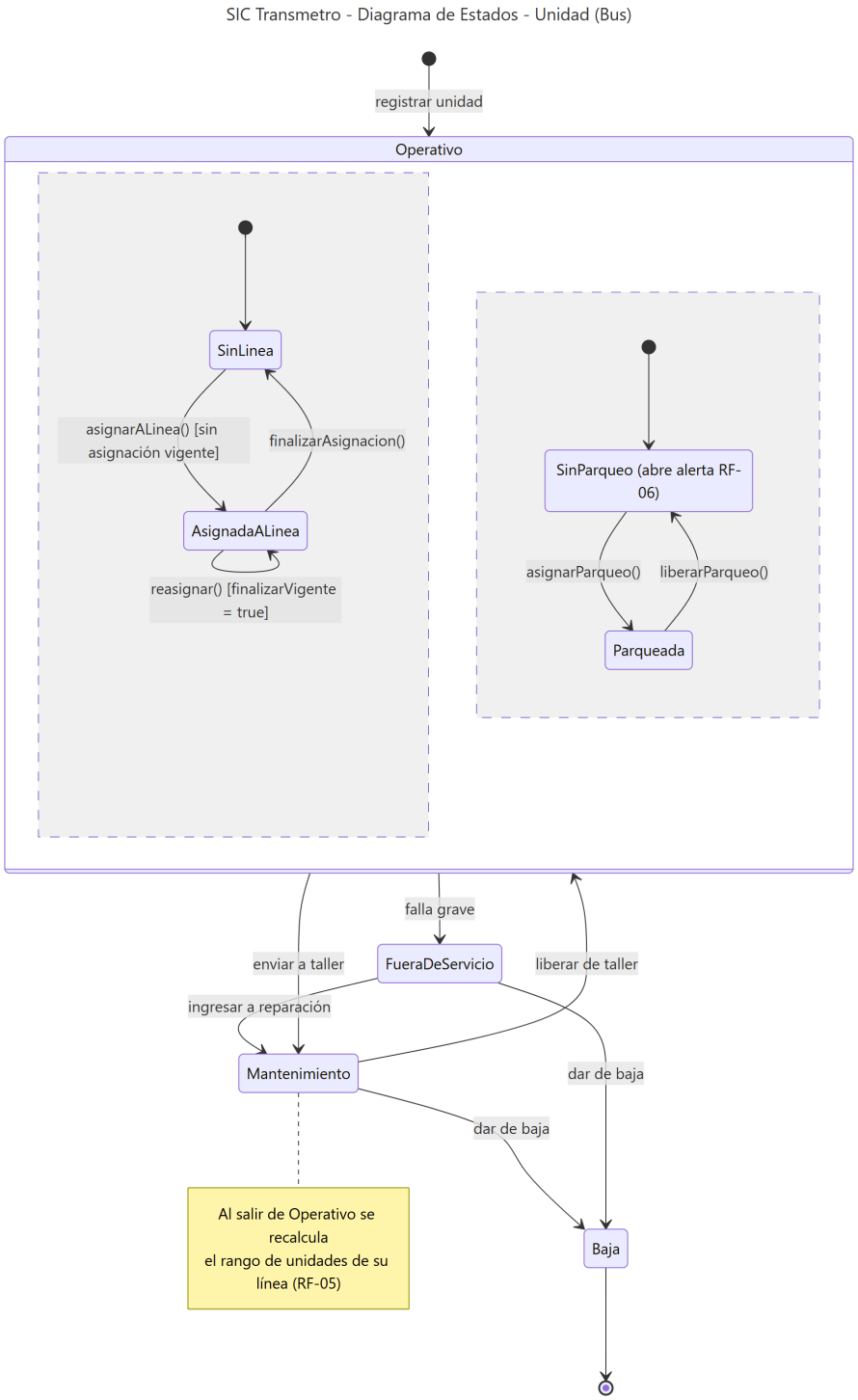


Figura 19. Estados de la unidad (bus)

Mientras está en el estado Operativo, la unidad tiene dos regiones concurrentes: su asignación a línea y su parqueo. Si sale de Operativo, se recalcula el rango de unidades de su línea (RF-05).

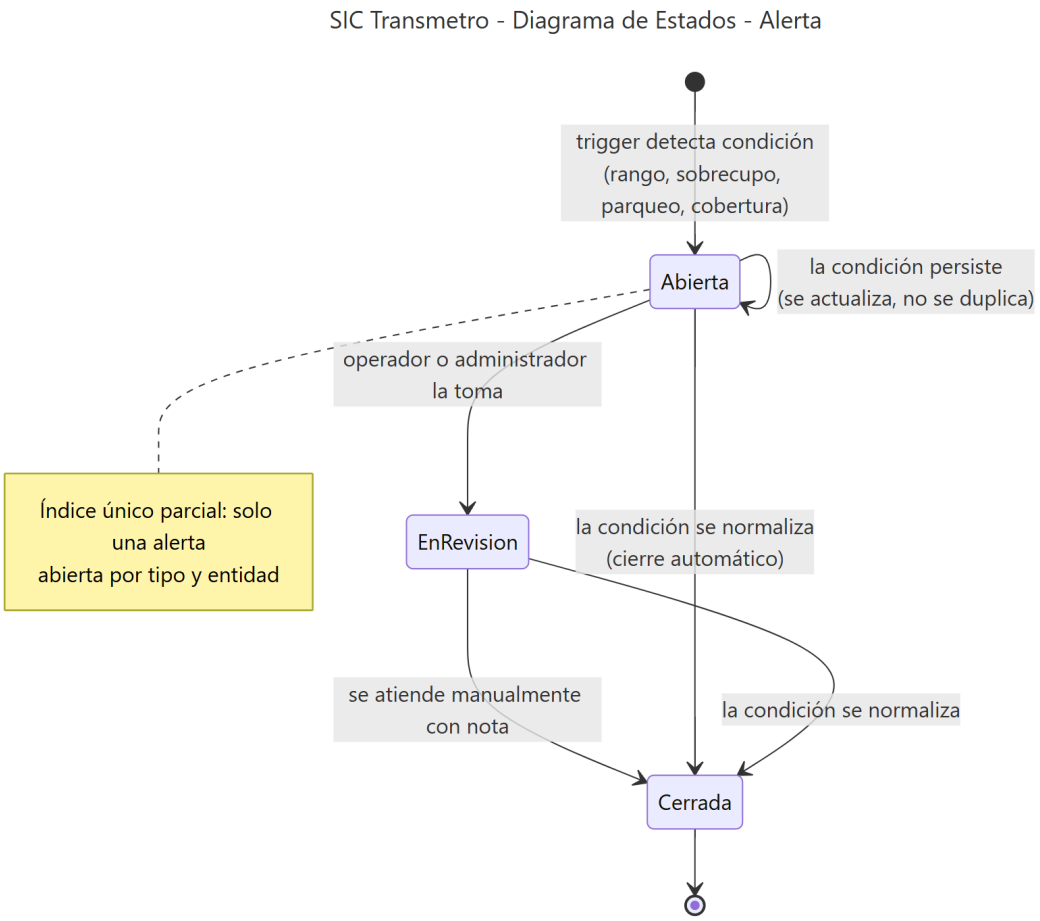


Figura 20. Estados de la alerta

Una alerta se cierra de dos formas: manualmente, con una nota, o automáticamente cuando la condición se normaliza. Solo puede haber una alerta abierta por tipo y por entidad.

3.6 Diagrama de componentes

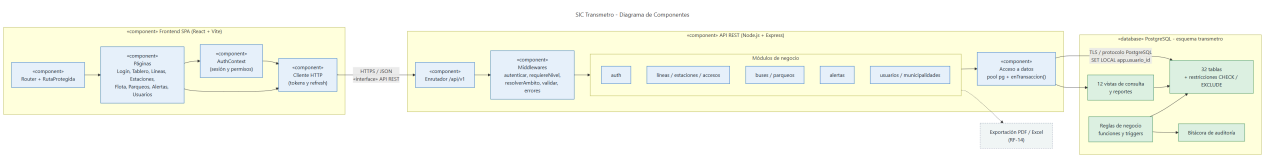


Figura 21. Diagrama de componentes

Las tres capas son independientes: el frontend solo conoce la interfaz REST y la API solo conoce la base. Esto cumple el requisito no funcional de capas desacopladas.

3.7 Diagrama de despliegue

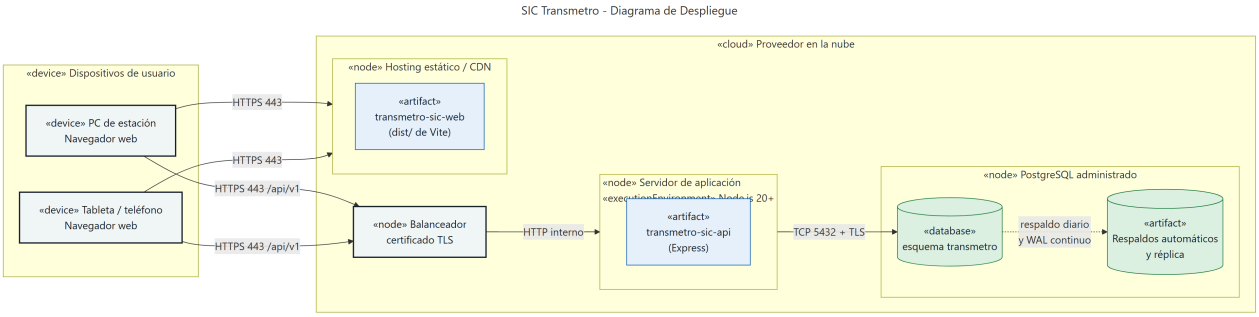


Figura 22. Diagrama de despliegue

Toda la comunicación externa va por HTTPS en el puerto 443, y la conexión a la base usa TLS. El servicio administrado de PostgreSQL hace respaldos diarios y guarda el WAL de forma continua, lo que cubre el requisito de respaldo y recuperación ante desastres.